

Cheat Sheet · GitHub Copilot

Ahorro de tokens y créditos · estilo apuntes · guía de uso diario

Idea fuerza: envía menos ruido, conserva solo el contexto útil y elige el modo, modelo y herramientas adecuados.

CONTEXTO > PROMPT

SALIDA CORTA = AHORRO

MODELO CORRECTO = ROI

Mapa mental rápido

1. Empezar una nueva sesión / Mantenerla

Mantén la sesión si refinas el mismo bloque, corriges un bug paso a paso y el contexto sigue limpio: así reutilizas información. Cada prompt vuelve a reunir el contexto, por lo que errores, logs y datos irrelevantes reducen esa eficiencia. Empieza otra si el agente se atasca, repite errores, arrastra logs obsoletos o cambia la etapa de producción.

2. Recupera JIT

Lee justo a tiempo símbolos, funciones, diff, selección y errores concretos. Evita carpetas enteras, archivos enormes o cientos de líneas de log si no aportan señal.

3. Saca lo estable del chat

Mueve arquitectura, reglas y convenciones a AGENTS.md, *.instructions.md, SKILL.md y *.prompt.md. Se versionan, reutilizan y evitan repetir contexto estable.

4. Output corto, mejor

Pide un diff mínimo, una línea final, YAML o un máximo de 6 bullets. La salida y el razonamiento innecesario pueden dominar el coste.

5. Modelo según tarea

Nano/mini/flash para clasificación, extracción y mecánica. Modelos potentes y mayor thinking effort solo para arquitectura, ambigüedad o debugging difícil.

6. Mide para decidir

Usa /context para inspeccionar lo arrastrado. /compact resume sesiones largas: reduce el contexto enviado y, por tanto, la cantidad que debe procesarse o cachearse; puede perder detalle, así que guarda lo estable en archivos.

7. Elimina el overhead

Desactiva tools y servidores MCP no críticos. Prioriza una llamada CLI estructurada sobre varias rondas de herramientas; reactiva lo necesario cuando cambie la tarea.



Ecosistema de personalización

Dónde vive cada instrucción y qué problema resuelve

El Ecosistema de Personalización de GitHub Copilot



Archivo	Dónde montarlo	Propósito + ejemplo mínimo
AGENTS.md	Raíz del repo; opcionalmente otro en una subcarpeta para alcance local.	Identidad, stack, comandos y reglas transversales. Run tests: <code>pnpm test</code> Never edit <code>dist/</code>
<code>.github/copilot-instructions.md</code>	Carpeta <code>.github/</code> .	Convenciones globales que Copilot aplica al proyecto. Use <code>TypeScript strict</code> . Prefer <code>small diffs</code> .
<code>*.instructions.md</code>	Normalmente <code>.github/instructions/</code> ; limita su alcance con <code>applyTo</code> .	Reglas por patrón de archivos. <code>applyTo: "**/*.test.ts"</code> Use <code>Arrange-Act-Assert</code> .
SKILL.md	Dentro de la carpeta de una skill, junto a scripts y referencias.	Flujo especializado, repetible y con herramientas. When debugging CI: 1. Read logs 2. Patch 3. Test
<code>*.prompt.md</code>	Normalmente <code>.github/prompts/</code> .	Comando/prompt invocable para una tarea frecuente. <code>Review #selection</code> . <code>Return risks + minimal diff</code> .
<code>*.agent.md</code>	Normalmente <code>.github/agents/</code> .	Persona, tools y flujo de un agente especializado; puede hacer <code>handoff</code> . Role: <code>test reviewer</code> Tools: <code>read</code> , <code>search</code>

Regla de montaje

Empieza por el archivo más simple y de menor alcance. AGENTS/instructions = reglas permanentes; prompt = petición reutilizable; skill = procedimiento con recursos; agent = rol y orquestación. Evita duplicar la misma regla en varios sitios.



Apuntes operativos del día a día

Checklist de 20 segundos antes de lanzar un prompt

Checklist

- ¿Estoy en el modo correcto: Ask, Plan o Agent?
- ¿Basta #selection, #problems o el símbolo exacto?
- ¿La sesión actual sigue limpia y relevante?
- ¿Uso el modelo y thinking effort mínimos suficientes?
- ¿Puedo pedir solo diff, YAML o máximo 6 bullets?
- ¿Están activas únicamente las tools necesarias?

Atajo: selección de terminal

Pasa solo el fragmento útil con #terminalSelection. Conserva el error, unas líneas previas y la traza relevante; no pegues 500 líneas.

Observabilidad express

Revisa input, output, cached input, model turns y tool calls. Si crecen sin mejorar el resultado, limpia contexto, baja el effort o reduce tools.

Restore Checkpoint

Dentro de la misma sesión, vuelve a un checkpoint anterior para descartar una rama fallida y recuperar archivos/estado del chat de ese punto. Úsalo antes de seguir refinando sobre cambios incorrectos. Precaución: revisa qué cambios posteriores se perderán y conserva aparte lo valioso.

Nueva sesión vs. Restore

Restore si el objetivo sigue siendo el mismo y conoces un estado bueno. Nueva sesión si el propio contexto está contaminado, la fase del trabajo cambió o quieres partir de un resumen limpio.

Modo	Cuándo	Coste relativo
Ask	Pregunta o explicación; sin edición ni bucle de tools.	Bajo
Plan	Investigar y diseñar antes de implementar.	Intermedio
Agent	Editar, ejecutar terminal y verificar hasta cumplir criterios.	Alto: más turns y tools

Más señal, menos contexto

✓ función concreta · ✓ diff · ✓ símbolo preciso · ✓ error relevante | × carpetas enteras · × logs infinitos · × historial sucio · × MCPs sin usar



Comandos y CLI: hoja rápida

Los comandos disponibles cambian según la superficie y extensiones instaladas; comprueba el autocompletado de / en tu chat.

Comando / acción	Para qué sirve	Cuándo usarlo
/context	Inspecciona el contexto que arrastra la sesión.	Antes de decidir si limpias, compactas o abres sesión.
/compact	Resume una conversación larga para reducir contexto futuro.	Cuando el historial es útil pero demasiado grande; valida que no pierda decisiones críticas.
/clear o nueva sesión	Parte sin el historial anterior.	Contexto contaminado, logs obsoletos o cambio de fase.
/help	Lista ayuda/comandos soportados en la superficie actual.	Cuando un slash command no aparece o dudas de su sintaxis.
Show Agent Debug Logs	Muestra turns, tokens y tool calls.	Auditar costes, bucles o llamadas innecesarias.
Compress Output	Reduce la salida de terminal/tools antes de pasarla al modelo.	Comandos verbosos, tests y logs extensos.

Eliminar el overhead

Prioriza CLI estructurada

Una única ejecución con salida JSON suele ser más barata y auditable que varias búsquedas MCP. Pide campos concretos o filtra la salida antes de devolverla al modelo.

Desactiva lo no crítico

Apaga temporalmente servidores MCP, integraciones y tools que no intervienen. Menos catálogo de herramientas = menos selección, menos llamadas accidentales y menos contexto auxiliar.

Ejemplo: comprobar el resultado de archivar un cambio OpenSpec

Run pwsh command: openspec list --json

Después de ejecutar /opsx-archive enable-cors, solicita una sola comprobación CLI y devuelve únicamente el elemento enable-cors, su estado y cualquier error. Si el CLI ya ofrece JSON, evita encadenar búsquedas o cargar documentación completa.

Patrón reusable

Ejecuta: <comando> --json

Filtra: <id/campos>

Devuelve: estado + error, máximo 3 bullets

No uses MCP salvo que el CLI no tenga el dato.



Comandos y CLI: referencias

Las referencias con almohadilla adjuntan contexto concreto al prompt. Escribe # y usa el autocompletado: los nombres disponibles pueden variar según IDE, versión y extensiones.

Referencia	Qué incorpora	Ejemplo de prompt
#selection	El código seleccionado en el editor.	Revisa #selection. Devuelve causa y diff mínimo.
#terminalSelection	Solo las líneas seleccionadas del terminal.	Diagnostica #terminalSelection. Máximo 4 bullets.
#file	Un archivo concreto elegido desde el chat.	Compara #file con #selection; no leas el resto.
#problems	Errores y avisos del panel Problems.	Agrupar #problems por causa raíz y prioriza el bloqueo.
#codebase	Búsqueda contextual sobre el proyecto.	Localiza en #codebase dónde se valida CORS.
#symbol	Una clase, función o símbolo concreto.	Explica llamadas y efectos de #symbol.
#changes / diff	Cambios actuales o diff, si la superficie lo ofrece.	Revisa #changes: regresiones, seguridad y tests faltantes.
#testFailure	Información del test fallido, cuando está disponible.	Usa #testFailure y propone el fix mínimo.

Cuándo usar

Cuando conoces exactamente la señal necesaria. Una referencia precisa evita que el agente explore archivos, terminales o diagnósticos que no aportan valor.

Precaución

No combines #codebase con muchos archivos y logs por defecto. Empieza con #selection o #problems y amplía solo si falta una dependencia real.

Patrón copy/paste

Usa solo #selection y #problems.
Objetivo: <resultado>.
No explores #codebase salvo bloqueo.
Devuelve: causa + diff mínimo + validación.



Modelos: coste y caso de uso

Regla práctica: mira el precio de salida y ajusta el razonamiento; el modelo más potente no siempre produce el mejor ROI.

Familia	Úsala para	Evita
Nano / mini / flash / haiku	Clasificación, extracción, formato, cambios mecánicos y primeras pasadas.	Arquitectura ambigua o debugging multicausa.
Codex / Sonnet / Pro	Coding general, refactorors acotados, tests y escritura técnica.	Microtarefas que resuelve un modelo barato.
Modelos frontier / Opus	Decisiones difíciles, arquitectura y debugging profundo.	Usarlos por defecto sin medir calidad/coste.

Thinking effort: cómo establecerlo

LOW

Tarea directa, poco ambigua y verificable: formato, rename, extracción, cambio pequeño. Empieza aquí si no necesitas explorar alternativas.

MEDIUM

Coding general, revisión y debugging normal. Buen punto de partida cuando Low omite dependencias o la tarea tiene varios pasos.

HIGH

Problemas difíciles, varias hipótesis, refactor amplio o arquitectura. Cuesta más tiempo/tokens: exige criterios de salida y verificación.

XHIGH

Máxima deliberación para casos realmente complejos y de alto impacto. Úsalo de forma excepcional; mide si mejora el resultado frente a High.

Cómo pedirlo

En el selector/configuración del agente establece Low, Medium, High o Xhigh si esa superficie y modelo lo permiten. No todos los modelos exponen todos los niveles. Patrón: comienza con el menor nivel razonable, sube uno si falla por razonamiento y vuelve a bajar para tareas mecánicas posteriores.

Ojo

Los tokens internos de razonamiento pueden aumentar el coste y la latencia. Thinking alto no sustituye contexto limpio, herramientas adecuadas ni tests.



Plantillas copy/paste

1) Debug barato y enfocado

Analiza solo `#selection` o el símbolo exacto.

Objetivo: causa raíz + fix.

Devuelve: 1) causa (máx. 2 bullets) 2) diff mínimo 3) dos tests.

No leas más archivos salvo necesidad.

2) Selección de terminal

Toma solo `#terminalSelection`.

Devuelve: causa probable, siguiente validación y fix mínimo.

Máximo 5 bullets.

3) Nueva sesión limpia

Estado válido: stack / error reproducible / hipótesis descartadas / archivos / siguiente paso.

Trabaja solo con esta base; pide contexto solo si bloquea.

4) Chain of Draft + caveman

Resuelve paso a paso en modo caveman: notas telegráficas, sin prosa, máximo 5 palabras por paso.

Después, respuesta final clara en 1 línea.

Finalidad: conservar el esqueleto útil del razonamiento reduciendo palabras, latencia y salida frente a una explicación extensa.

Prompts de control rápido

Salida mínima

Devuelve únicamente el diff y 3 riesgos.
Sin introducción ni recapitulación.

CLI antes que MCP

Usa el CLI disponible y salida JSON.
Recorre a MCP solo si falta el dato.

Checkpoint seguro

Antes de restaurar, enumera qué cambios posteriores se perderán y qué debo conservar.

Compactación

Antes de `/compact`, resume decisiones, restricciones y próximos pasos en un archivo estable.

Regla de oro

Eleva la señal. Menos ruido = más precisión + menos coste. Mantén el contexto mientras sea correcto y útil; límpialo cuando empiece a dirigir al agente hacia el pasado.



Archivos frecuentemente utilizados

Plantillas completas para copiar, pegar y adaptar. Sustituye los marcadores <<SPECIFY>> por los datos reales del proyecto.

1. Reglas del agente

AGENTS.md · Contexto y reglas transversales del repositorio.

```
# You are assisting in a <<SPECIFY_LANGUAGE_PROGRRAMING_PROJECT>>

Stack: <<SPECIFY_LANGUAGE_PROGRRAMING_PROJECT>>

## Rules
- Diagnose before editing. Return root cause first unless told to skip.
- Minimal changes only. Do not refactor outside the task scope.
- Show only modified method or block – not full file rewrites.
- No changes unless explicitly requested.
- No tests unless explicitly requested.
- No comments, JSDoc, or docstrings unless asked.
- No suggestions beyond the task scope. State change. Show fix. Stop.
- Limit analysis to max(4 bullets).

## Output
- Drop filler: no "Sure!", "Happy to help", "Of course", "Certainly".
- Code blocks unchanged and copy-paste safe.
- No --- dashes or decorative Unicode.

## Model selection
- Mechanical tasks (rename, boilerplate, docstring): use smallest available model.
- Exploration and synthesis (diagnose, refactor, explain): use standard model.
- Architecture decisions, hard bugs: use the most capable model only if needed.

## Test commands
- All tests: <<SPECIFY>>
- Orders only: <<SPECIFY>>
- Shipping only: <<SPECIFY>>
- Notifications only: <<SPECIFY>>
```

Dónde ubicarlo	Uso / invocación	A qué afecta	Propósito
Raíz del repo o subcarpeta.	Pasivo. Abre copilot en el repo y escribe el prompt normal. No hace falta decir «usa AGENTS.md».	A la carpeta donde está y sus descendientes; prevalece el más cercano.	Contexto y reglas transversales del repositorio.

Cómo usarlo

Colócalo en la raíz. En monorepos puedes añadir otros AGENTS.md dentro de subcarpetas; el más cercano al archivo trabajado tiene prioridad. Esta página reproduce el contenido que proporcionaste.



2. Ajustes del workspace

.vscode/settings.json · Configuración compartida de VS Code y GitHub Copilot.

```
{
  "github.copilot.chat.localeOverride": "en",
  "chat.tools.compressOutput.enabled": true,
  "chat.checkpoints.enabled": true,
  "github.copilot.chat.codeGeneration.useInstructionFiles": true,
  "chat.includeApplyingInstructions": true,
  "chat.useAgentSkills": true,
  "chat.promptFilesLocations": {
    ".github/prompts": true
  },
  "chat.agentFilesLocations": {
    ".github/agents": true
  },
  "chat.agentSkillsLocations": {
    ".github/skills": true
  }
}
```

Dónde ubicarlo	Uso / invocación	A qué afecta	Propósito
.vscode/settings.json	Pasivo. Se aplica al abrir el workspace. No se escribe en el prompt. Si no se refleja, ejecuta Developer: Reload Window.	A VS Code/Copilot dentro de este workspace. Para todos tus proyectos, usa Settings (JSON) de usuario.	Configuración compartida de VS Code y GitHub Copilot.

Cómo usarlo

localeOverride fuerza las respuestas de Copilot en inglés. compressOutput comprime salidas grandes del terminal antes de enviarlas al modelo. El resto activa checkpoints y el descubrimiento de instructions, prompts, agents y skills en las ubicaciones usadas por este documento.



3. Instrucciones por ruta

.github/instructions/typescript.instructions.md · Reglas automáticas para archivos TypeScript.

```
---
applyTo: "**/*.ts,**/*.tsx"
---

# TypeScript instructions

- Use TypeScript strict mode. Do not introduce `any`.
- Preserve the existing architecture and public API.
- Prefer small functions and early returns.
- Make the smallest change that satisfies the task.
- Do not edit generated files, `dist/`, or lockfiles unless required.
- Keep imports sorted according to the repository formatter.
- Reuse existing utilities before creating new abstractions.

## Tests

- Follow Arrange-Act-Assert.
- Test observable behavior, not implementation details.
- Add or change only tests related to the requested behavior.
- Run the narrowest relevant test command first.

## Output

- Return the root cause before the patch when diagnosing.
- Show the minimal diff and the validation performed.
- Limit the final summary to 4 bullets.
```

Dónde ubicarlo	Uso / invocación	A qué afecta	Propósito
.github/instructions/ NAME.instructions.md	Pasivo. Prompt normal sobre un archivo que coincida. No se menciona el archivo de instrucciones.	Solo a rutas que coincidan con applyTo.	Reglas automáticas para archivos TypeScript.

Cómo usarlo

Cambia applyTo para limitar el alcance. Puedes separar testing, frontend o documentación en distintos archivos *.instructions.md. Se aplican automáticamente cuando coincide la ruta.



4. Skill especializada

.github/skills/focused-bug-fix/SKILL.md · Procedimiento reutilizable que Copilot carga cuando es relevante.

```
---
name: focused-bug-fix
description: >-
  Diagnose reproducible failures with minimal fixes. Use for errors,
  failing tests, or broken behavior.
---

# Focused bug fix

## Workflow

1. Read only the error, selected code, and directly related symbols.
2. Reproduce the failure with the narrowest available command.
3. State the root cause in no more than 2 bullets.
4. Apply the smallest change that fixes that root cause.
5. Run the focused test or validation again.
6. Stop if broader refactoring is not required.

## Context rules

- Prefer `#selection`, `#problems`, or `#terminalSelection`.
- Do not load folders or full logs by default.
- Ask for one missing dependency at a time.

## Output

- Root cause
- Minimal patch
- Validation result
- Remaining risk, only if real

Do not add unrelated improvements or documentation.
```

Dónde ubicarlo	Uso / invocación	A qué afecta	Propósito
.github/skills/ focused-bug-fix/SKILL.md	Automática si es relevante. En CLI: /focused-bug-fix corrige X. Consulta /skills list.	A la tarea que activa la skill; no depende de una ruta applyTo.	Procedimiento reutilizable que Copilot carga cuando es relevante.

Cómo usarlo

El nombre de la carpeta y el campo name deben describir una capacidad concreta. Añade scripts o referencias en la misma carpeta solo cuando el flujo los necesite.



5. Prompt reutilizable

`.github/prompts/focused-review.prompt.md` · Comando manual invocable desde el chat.

```
---
name: focused-review
description: Review selected code and return only actionable findings.
agent: ask
argument-hint: "focus=correctness|security|performance"
---
```

Review only ``#selection`` and the directly related diagnostics in ``#problems``.

Focus: `${input:focus:correctness}`

Check for:

- incorrect behavior or edge cases
- security or data-loss risks
- unnecessary complexity
- missing validation directly related to the selection

Rules:

- Do not inspect the whole codebase unless the selection cannot be understood.
- Do not suggest stylistic refactors.
- Do not rewrite the full file.
- Rank findings by severity.

Return:

1. Findings: maximum 4 bullets
2. Minimal diff, only when a fix is needed
3. One validation command

Dónde ubicarlo	Uso / invocación	A qué afecta	Propósito
<code>.github/prompts/focused-review.prompt.md</code>	Manual. En el chat del IDE: <code>/focused-review focus=security</code> . No hace falta pedir «usa prompt.md».	Solo al prompt invocado y al contexto/referencias que adjunte.	Comando manual invocable desde el chat.

Cómo usarlo

Guárdalo en `.github/prompts/` e invócalo como `/focused-review`. Los campos admitidos y las referencias `#` pueden variar según la versión y la superficie.



6. Instrucciones globales de Copilot

github/copilot-instructions.md · Convenciones automáticas para todo el repositorio.

```
# Repository instructions

## Project

- Runtime: <<SPECIFY_RUNTIME_AND_VERSION>>
- Stack: <<SPECIFY_FRAMEWORKS_AND_DATABASE>>
- Package manager: <<SPECIFY>>
- Source code: `src/`
- Tests: `tests/`

## Working rules

- Diagnose before editing and state the root cause first.
- Make the smallest change that satisfies the request.
- Preserve public APIs unless the task explicitly changes them.
- Reuse existing patterns and utilities.
- Never edit generated files or `dist/`.
- Do not add dependencies without explicit approval.

## Commands

- Install: <<SPECIFY>>
- Build: <<SPECIFY>>
- Lint: <<SPECIFY>>
- Test all: <<SPECIFY>>
- Test one: <<SPECIFY>>

## Definition of done

- Relevant tests pass.
- Formatting and lint checks pass.
- The final response states changed files and validation.
```

Dónde ubicarlo	Uso / invocación	A qué afecta	Propósito
.github/ copilot-instructions.md	Pasivo. CLI: ejecuta copilot desde el repo y formula el prompt normal. /init puede generar una base.	A todo el repositorio y a todas las solicitudes compatibles.	Convenciones automáticas para todo el repositorio.

Cómo usarlo

Úsalo para datos y reglas útiles en casi todas las tareas: stack, estructura, comandos validados y definición de terminado. Evita incluir procedimientos largos o instrucciones específicas de una sola ruta.



7. Agente personalizado

.github/agents/focused-implementer.agent.md · Rol especializado con comportamiento propio.

```
---
name: focused-implementer
description: Implement a well-scoped change with minimal edits and verification.
argument-hint: "Describe the requested change and acceptance criteria"
---
```

Focused implementer

You implement only the requested behavior.

Workflow

1. Restate the acceptance criteria in no more than 3 bullets.
2. Inspect only directly related files and symbols.
3. Identify the smallest safe implementation.
4. Apply the change without unrelated refactoring.
5. Run the narrowest relevant validation.
6. Report changed files, validation, and any real remaining risk.

Guardrails

- Ask before adding dependencies or changing public APIs.
- Do not modify generated files.
- Do not broaden the task.
- Keep the final response under 6 bullets.

Dónde ubicarlo	Uso / invocación	A qué afecta	Propósito
.github/agents/ focused-implementer.agent. md	Manual o delegado. CLI: /agent. Prompt: «Usa focused-implementer para X». IDE: selector de agentes.	A la tarea y archivos que gestione ese agente; no se aplica pasivamente a todo el repo.	Rol especializado con comportamiento propio.

Cómo usarlo

En VS Code selecciónalo en el desplegable de agentes. En Copilot CLI escribe /agent y elígelo. También puedes poner en el prompt: Usa focused-implementer para implementar X.



Anatomía de un proyecto Python + Copilot

Una estructura limpia separa código, pruebas, documentación y personalización del agente.

```
mi-proyecto/
|-- .vscode/
|   |-- settings.json
|-- .github/
|   |-- agents/
|   |   |-- focused-implementer.agent.md
|   |-- instructions/
|   |   |-- python.instructions.md
|   |-- prompts/
|   |   |-- focused-review.prompt.md
|   |-- skills/
|   |   |-- caveman/
|   |   |   |-- SKILL.md
|   |   |-- copilot-instructions.md
|-- data/
|   |-- raw/
|   |-- processed/
|-- docs/
|-- logs/ [LOCAL]
|-- src/
|   |-- mi_proyecto/
|   |   |-- __init__.py
|   |   |-- ...
|-- tests/
|-- .venv/ [LOCAL]
|-- AGENTS.md
|-- .gitignore
|-- README.md
|-- Makefile
|-- pyproject.toml
|-- requirements.txt [OPTIONAL]
```

SE VERSIONA

.vscode/settings.json, .github/, src/, tests/, documentación y archivos Markdown. Así el equipo comparte ajustes, código, reglas y flujos.

NO SE VERSIONA

.env/, logs generados, cachés, secretos y artefactos. Añádelos a .gitignore. En data/, ignora datos pesados o sensibles.

COPILLOT

Usa .github/ como ubicación compartida. .agents/skills también es válida para skills, pero elige una sola ubicación para evitar duplicados.

PYTHON

pyproject.toml centraliza build y herramientas. requirements.txt es opcional si tu gestor ya bloquea o declara dependencias.

Correcciones sobre la idea inicial

- Makefile no lleva extensión .md.
- Añade AGENTS.md en la raíz y .vscode/settings.json para ajustes compartidos.
- Dentro de src/ crea el paquete importable mi_proyecto/.
- Una skill necesita su carpeta y un archivo SKILL.md.
- logs/ y .venv/ pueden existir localmente, pero deben quedar fuera de Git.

Regla sencilla

Versiona lo necesario para recrear, entender y validar el proyecto. Ignora lo generado, pesado, secreto o específico de una máquina.